

Task Scheduler Performance Optimization: Algorithmic Rewrite, Query-Based Prefetch & Job-Paginated Page Background Task Loading

Business Central AL extension — DHTMLX Scheduler control add-in
(page 50621 "DHX Scheduler (Project)", caption "Task Scheduler")

Scope	Section/event JSON generation for the Task Scheduler's weekly timeline (Job/Job Task hierarchy + Day Planning events)
Objects touched	Codeunit 50604, Page 50621, Control Add-in interface + wrapper.js
Objects added	Codeunit 50720 "Task Scheduler BG Sections", Test Codeunit 60029 "Task Scheduler Sections Perf"
Platform feature used	Business Central Page Background Tasks (child-session async compute) — same mechanism previously proven on the Gantt add-in's resource panel
Related prior work	Builds directly on the pattern documented in Gantt-Sync-to-Async-PageBackgroundTask-Enhancement.pdf (page 50620); Sections 6–7 of this document reuse that document's stash-and-poll and control-add-in-callback findings without re-deriving them
Prepared	2026-09-01

Contents

1. Executive Summary

2. Problem Statement

3. Solution Architecture — Part A: algorithmic rewrite (no schema/behavior change)

4. Solution Architecture — Part B: Job-paginated Page Background Task loading

5. Implementation Detail — Codeunit 50604 "DHX Data Handler"

6. Implementation Detail — Page 50621 & Codeunit 50720 (background task target)

7. Implementation Detail — Control Add-in & wrapper.js

8. Production Bug Found During Live Verification: "Index out of bounds"

9. Verification

10. Object Inventory

11. Known Limitations & Follow-ups

12. Appendix A — Page Background Task API Reference

13. Appendix B — Glossary of Terms

14. Appendix C — How to Verify This Yourself

1. Executive Summary

Page 50621 "DHX Scheduler (Project)" (caption "Task Scheduler") renders a DHTMLX Scheduler timeline backed by Business Central data: one row ("section") per Job Task touched by a Day Planning in the visible week, grouped under its owning Job and nested to match the Job Task WBS hierarchy. Unlike the Gantt add-in's Job Task tree (which scans the whole Job Task table regardless of date), Task Scheduler's dataset is already date-bounded to one week — so the slowness reported against this page was not a "too much data" problem, it was an algorithmic one.

Two distinct defects were found and fixed in `codeunit_50604_DHXDataHandler.al`'s section-tree builder:

- **A multi-pass ancestor-resolution loop.** `AddAncestorsToTemp` re-scanned its entire working set on every pass of a `repeat...until not NewAncestorAdded` outer loop, issuing a fresh filtered `FindLast()` query for every indented task on every pass — an $O(\text{passes} \times \text{tasks} \times \text{query})$ cost that grows with both WBS depth and job count.
- **N+1 per-row record access.** The main Day Planning loop called `JobTasks.Get()`, `Resource.Get()` (twice — assigned and requested), and `Vendor.Get()` once per Day Planning row instead of once per distinct value.

Both were rewritten: ancestor resolution now does one sorted pass per distinct Job No. held in memory, and Resource/Vendor lookups are served from bulk-prefetched dictionaries built with a single OR-filtered query each.

On top of that algorithmic fix, a second, independent enhancement mirrors the pattern already proven on the Gantt add-in's resource panel: the section tree is now built and rendered in two tiers. The first ~50 rows (accumulated at Job boundaries — a Job is never split across the boundary) render synchronously, exactly as before. Any remaining Jobs are fetched by a Page Background Task and appended into the already-rendered timeline once ready, using the same stash-and-poll delivery mechanism documented for the Gantt resource panel. This applies uniformly to the page's three data-loading entry points — initial `ControlReady`, `RefreshSchedule` (the Previous/Today/Next/date-jump actions), and `OnTimelineNavigate` — via one shared local procedure, so none of them had to reinvent the pagination or background-task logic independently.

Live verification against a real Business Central sandbox (CRONUS NL) surfaced a genuine production bug in the rewritten ancestor-resolution code — a fixed-size array indexed by WBS depth that threw `Index out of bounds` against real data — caught, root-caused, and fixed before this document was written. Section 8 documents it in full, including the exact AL call stack captured from the live error dialog. Both the fix and the surrounding feature are confirmed working end-to-end: clean compile/build, three passing correctness tests, and a live re-test (initial load and a Next-navigation click) with no crash and no console error.

2. Problem Statement

Before this change, all three of page 50621's data-loading entry points called the non-paginated `GetYUnitElementsJSON_Project` family directly and synchronously, blocking the whole AL trigger — and by extension the browser — until the entire section tree and event list for the requested week were built and returned:

BEFORE — `src/codeunit/codeunit_50604_DHXDataHandler.al` (conceptual shape, both hot spots)

```
// Hot spot 1: repeated multi-pass ancestor resolution
repeat
    NewAncestorAdded := false;
    // snapshot ALL of TEMPJobTasks again, every single pass
    if TempSnapshot.FindSet() then
        repeat
            if TempSnapshot.Indentation > 0 then begin
                JobTaskReal.SetRange("Job No.", TempSnapshot."Job No.");
                JobTaskReal.SetFilter("Job Task Type", '<>%1&<>%2&<>%3', ...);
                JobTaskReal.SetFilter("Job Task No.", '<%1', TempSnapshot."Job Task No.");
                JobTaskReal.SetRange(Indentation, TempSnapshot.Indentation - 1);
                if JobTaskReal.FindLast() then // ← one fresh query PER indented task, PER pass
                    ...
            end;
            until TempSnapshot.Next() = 0;
        until not NewAncestorAdded;

// Hot spot 2: N+1 record access in the main Day Planning Loop
if DayPlanning.FindSet() then
    repeat
        JobTasks.Get(DayPlanning."Job No.", DayPlanning."Job Task No."); // ← 1 Get() per row
        if Resource.Get(DayPlanning."Assigned Resource No.") then ... // ← 1 Get() per row
        if Resource.Get(DayPlanning."Requested Resource No.") then ... // ← 1 Get() per row
        if Ven.Get(DayPlanning."Vendor No.") then ... // ← 1 Get() per row
    until DayPlanning.Next() = 0;
```

Neither of these costs scales with the size of the actual response the user sees — a week with 5 distinct Job Tasks nested 3 levels deep could still trigger 3 outer passes × up to 5 `FindLast()` calls each in the ancestor resolver alone, before a single event row was even read.

OBSERVED IMPACT

Every page open and every Previous/Today/Next/date-jump click paid the full cost of both hot spots, synchronously, blocking the browser until the whole response was built — regardless of how few sections the visible week actually contained. The deeper a company's WBS structure and the more concurrent Jobs it tracks, the worse this scaled, independent of how "busy" any single week was.

3. Solution Architecture — Part A: Algorithmic Rewrite

Part A changes only *how* the existing result is computed — the JSON shape, field names, and ancestor-insertion contract are unchanged, so every caller of `GetYUnitElementsJSON_Project` benefits with no signature change.

3.1 Ancestor resolution: one sorted pass per Job, not a repeated full rescan

The rewritten `AddAncestorsToTemp` collects the distinct Job Nos among the originally-touched tasks (one in-memory pass, no query), then for each Job does exactly one sorted `FindSet()` over that Job's own Job Task list, held in memory. Walking that list forward once, a small level-index structure tracks "the most recently seen ancestor-eligible row at level N" — the same value the old code re-derived via `FindLast()` for every task on every pass, now read off in $O(1)$ as each row is visited:

AFTER — `codeunit_50604_DHXDataHandler.al, AddAncestorsToTemp` (excerpt)

```
local procedure AddAncestorsToTemp(var TEMPJobTasks: Record "Job Task" temporary)
var
    ...
    // Dictionary, not a fixed-size array — see Section 8 for why this matters
    LastAncestorAtLevel: Dictionary of [Integer, Code[20]];
    LastAncestorCode: Code[20];
begin
    ... // snapshot touched tasks, collect distinct Job Nos (no query)

    foreach JobNo in DistinctJobNos do begin
        Clear(LastAncestorAtLevel);
        JobTaskReal.SetRange("Job No.", JobNo);
        if JobTaskReal.FindSet() then // ← exactly ONE query per Job, total
            repeat
                // Direct parent = Last ancestor-eligible row seen so far at Indentation-1
                if (JobTaskReal.Indentation > 0)
                    and LastAncestorAtLevel.Get(JobTaskReal.Indentation, LastAncestorCode) then
                    ParentOfMap.Add(JobTaskReal."Job Task No.", LastAncestorCode);

                if /* not Posting/End-Total/Total */ then
                    LastAncestorAtLevel.Set(JobTaskReal.Indentation + 1, JobTaskReal."Job Task No.");
            until JobTaskReal.Next() = 0;

            // Every originally-touched task of this Job, walked up ParentOfMap to its
            // root, inserting every missing ancestor — handles arbitrary nesting depth
            // in one climb, no outer "did anything change" loop needed.
            foreach StartKey in ToProcess do begin ... end;
        end;
    end;
end;
```

This preserves the exact original semantics — "direct parent = the last Begin-Total/Heading row (never Posting/End-Total/Total) strictly before this task's own Job Task No., at exactly Indentation – 1" — because a forward walk in ascending Job Task No. order visits candidate ancestors in exactly the order the old `FindLast()` query would have returned them; the new code just remembers the answer instead of re-querying for it.

3.2 Bulk prefetch instead of per-row Gets

Before the main Day Planning loop runs, every distinct Assigned/Requested Resource No. and Vendor No. referenced in the filtered period is collected into a `List of [Code[20]]`, then fetched in one `FindSet()` each against an OR-filter built from that list (`BuildCodeOrFilter`), populating `Dictionary of [Code[20], Text]` lookups. The main loop then reads from those dictionaries instead of calling `Resource.Get()` / `Vendor.Get()` per row. The already-touched-task check (`if not TEMPJobTasks.Get(...) then JobTasks.Get(...)`) applies the same idea to Job Task lookups — a task already seen this run is never re-fetched from the database.

4. Solution Architecture — Part B: Job-Paginated Page Background Task Loading

Part B is additive on top of Part A: the first ~50 section rows render synchronously as before (now much cheaper, thanks to Part A); any remainder is fetched off the interactive request path.

4.1 Why pagination is scoped at the Job boundary, not the row

Splitting output at an arbitrary row count risks cutting a Job's own indented task tree in half — rendering an orphaned sub-branch with no visible parent heading. Instead, the paginated builder accumulates whole Jobs (in the same Job No. order the tree-build already walks) until the running row total reaches the requested maximum; the Job that pushes the total over the line is still included in full, and every Job after it is deferred:

AFTER — codeunit_50604_DHXDataHandler.al, GetYUnitElementsJSON_Project_Paged (excerpt)

```
RunningTotal := 0;
foreach JN in JobOrder do
    if RunningTotal >= MaxRows then begin
        if RemainingJobFilter <> '' then
            RemainingJobFilter += '|';
        RemainingJobFilter += JN;           // deferred to the background task
    end else begin
        IncludedJobs.Add(JN, true);
        RunningTotal += JobRowCounts.Get(JN); // whole Job always included together
    end;
```

A page whose entire result already fits within `MaxRows` produces `RemainingJobFilter = ''` — and, by construction, output identical to calling the original non-paged procedure directly, since the tree-build loop itself (Section 5.3) is the same stack-based algorithm, merely pre-filtered to the included Jobs.

4.2 End-to-end flow — reusing the Gantt resource panel's proven mechanism

The enqueue → background compute → stash → poll → deliver sequence is structurally identical to the one documented for the Gantt add-in's resource panel (see the companion document referenced on the cover page), substituting Task Scheduler's own objects:

- 1 **Synchronous first page.** `ControlReady`, `RefreshSchedule`, and `OnTimelineNavigate` all call one shared local procedure, `LoadSchedulerSectionsPaginated`, which calls the Part-A-optimized `GetYUnitElementsJSON_Project_Paged` and renders the first page immediately, exactly as the non-paginated call used to.
- 2 **Enqueue.** If `RemainingJobFilter` is non-empty, the same procedure calls `CurrPage.EnqueueBackgroundTask(...)` against Codeunit 50720 "Task Scheduler BG Sections" and returns immediately — the trigger does not wait.
- 3 **Background child session.** Codeunit 50720 reads its parameters via `Page.GetBackgroundParameters()`, calls the same paginated builder scoped to the remaining Jobs only, and returns via `Page.SetBackgroundTaskResult()`.
- 4 **Completion callback (stash only).** `OnPageBackgroundTaskCompleted` stashes the result into plain page variables — it does not call the control add-in directly (Section 7 explains why not).
- 5 **Client poll.** wrapper.js's bounded poll timer (same 500 ms / 60-attempt shape as the Gantt resource panel) calls `OnPollSectionsResult`, which delivers the pending JSON into the control add-in via `CurrPage.DhxScheduler.AppendSections(...)` and stops polling.

4.3 Staleness guard

`OnPageBackgroundTaskCompleted` compares the completing task's scope (`AnchorDate` / `ResourceFilter` / `JobFilter` / `JobTaskFilter`) against the page's *current* state before promoting a pending result — a background fetch for last week's data that completes after the user has already clicked Next twice is discarded silently rather than being appended on top of the wrong week.

5. Implementation Detail — Codeunit 50604 "DHX Data Handler"

5.1 GetYUnitElementsJSON_Project_Paged — a deliberately self-contained sibling

`GetYUnitElementsJSON_Project_Paged` does not call the non-paged `GetYUnitElementsJSON_Project` internally — it is a self-contained procedure that shares the same Part-A fixes (bulk prefetch, single-pass ancestor resolution) but adds its own row-counting and Job-filtering bookkeeping. This was a deliberate choice: threading pagination state through the already-verified non-paged procedure risked introducing a regression into the code path every other potential future caller of the plain, unpaginated API still depends on. The two procedures are independently testable and independently correct; Section 9's test codeunit exercises the shared Part-A logic through both entry points.

5.2 Shared filter helper

`ApplyProjectSchedulerDayPlanningFilters` centralizes the "Plan Date" range plus Job/Job Task/Resource filter construction that both the paged and non-paged builders — and the bulk-prefetch scan that runs ahead of them — all need identically, so the three can never drift out of sync with each other.

5.3 Event/section correlation across the pagination boundary

Because Day Planning events are scanned once for the *entire* filtered period (not just the first page — the row-counting pass needs to see every touched task to decide the Job boundary correctly), each event is tagged with its owning Job No. in a list running parallel to the event array. After the Job-inclusion boundary is decided, that parallel list filters the event array down to only the events belonging to an included Job; the rest travel to the background task with their Jobs, not as a separate query.

6. Implementation Detail — Page 50621 & Codeunit 50720

6.1 One shared loader for three entry points

`ControlReady`, the local procedure `RefreshSchedule` (bound to the Previous/Today/Next actions and the date-lookup action), and `OnTimelineNavigate` (fired by the control add-in's own drag-scroll timeline navigation) previously each called the data-fetch procedures independently. All three now delegate to one shared `LoadSchedulerSectionsPaginated` procedure, so the pagination/enqueue/staleness-guard logic exists in exactly one place and cannot drift between the three call sites.

6.2 Codeunit 50720 "Task Scheduler BG Sections"

Structured identically to the Gantt add-in's Codeunit 50713 "Gantt BG Resource Panel Data": a plain codeunit whose `OnRun()` reads its scope via `Page.GetBackgroundParameters()`, calls back into Codeunit 50604's paginated builder for the remaining Jobs only, and returns the resulting `resourcesJson / eventsJson` pair via `Page.SetBackgroundTaskResult()`. Object ID 50720 was chosen clear of both the existing Gantt background-task codeunit (50713) and an unrelated, unmerged experimental codeunit on a separate branch (50714), to avoid any future merge collision.

7. Implementation Detail — Control Add-in & wrapper.js

7.1 Why the completion trigger only stashes, never calls the control add-in

This constraint was already discovered and documented against the Gantt add-in (see the companion document's Section 7): any `CurPage.<AddInName>.*` call issued directly from `OnPageBackgroundTaskCompleted / OnPageBackgroundTaskError` is rejected by BC Server with *"attempted to issue a client callback on an Automation object... disallowed callback was issued from the OnPageBackgroundTaskCompleted or OnPageBackgroundTaskError trigger."* Task Scheduler's `OnPageBackgroundTaskCompleted` was written to that already-proven constraint from the start — it stashes into page variables only, and delivery happens from `OnPollSectionsResult` instead, a normal JS-initiated synchronous trigger where control add-in calls are permitted.

7.2 AppendSections — an in-place refresh, not a full reset

wrapper.js's existing reload functions (`Init`, `RefreshTimeline`) both funnel through `RecreateTimelineView`, which unconditionally does `scheduler.deleteView("timeline") + scheduler.createTimelineView(...)` — a full section-tree replace — followed by `scheduler.clearAll()` + `scheduler.parse()`, a full event replace. Routing the background task's incremental result through either of those would discard the already-rendered first page and repaint everything, defeating the purpose of paginating in the first place.

Instead, the new `AppendSections(sectionsJson, eventsJson)` function is modeled directly on an existing, already-proven non-destructive pattern already present in this same file — `ToggleCollapseExpandAllSections()` — which mutates `scheduler.matrix.timeline.y_unit_original / y_unit` in place and fires `scheduler.callEvent("onOptionsLoad", [])` to force a re-render without ever calling `deleteView / createTimelineView / scheduler.init`. `AppendSections` follows the same shape for the new sections, then calls the additive (non-clearing) `scheduler.parse(newEvents)` for the new events — DHTMLX's `parse()` merges into existing data by default, so no `clearAll()` is needed for an append.

7.3 Poll timer — same shape as the Gantt resource panel, deliberately not reinvented

The poll interval (500 ms) and attempt ceiling (60 attempts $\approx$ 30 s) exactly match `wrapper.js`'s existing `NotifyResourcePanelTaskPending` implementation in the Gantt add-in. Reusing the same constants (rather than independently choosing new ones for Task Scheduler) keeps the two add-ins' async-loading behavior consistent for anyone maintaining both.

8. Production Bug Found During Live Verification: "Index out of bounds"

LIVE-TESTED FINDING

The first live re-test of the rewritten `AddAncestorsToTemp` against real CRONUS NL data crashed with a Business Central runtime error dialog reading **"Index out of bounds."** The section tree never rendered — the control add-in was left showing "LOADING / Please wait..." indefinitely behind the error dialog.

8.1 Root cause

The initial implementation of the level-index structure in Section 3.1 used a fixed-size array, not a dictionary:

BEFORE THE FIX — `codeunit_50604_DHXDataHandler.al`, `AddAncestorsToTemp`

```
LastAncestorAtLevel: array[60] of Code[20];
...
if (JobTaskReal.Indentation > 0) and (LastAncestorAtLevel[JobTaskReal.Indentation] <> '') then
    ParentOfMap.Add(JobTaskReal."Job Task No.", LastAncestorAtLevel[JobTaskReal.Indentation]);
...
LastAncestorAtLevel[JobTaskReal.Indentation + 1] := JobTaskReal."Job Task No.";
```

A fixed-size array indexed directly by a data-driven value (`Indentation`) is only as safe as the assumption that the value never exceeds the array's bound. Sixty levels appeared generous against any test fixture used during development — but CRONUS NL's real Job Task data exceeded it, and AL arrays throw rather than silently clamp on an out-of-range index.

8.2 Diagnosis

The exact AL call stack was captured live from the Business Central error dialog's "If requesting support" details panel:

```
AL call stack:
"DHX Data Handler"(CodeUnit 50604).AddAncestorsToTemp line 43
"DHX Data Handler"(CodeUnit 50604).GetYUnitElementsJSON_Project_Paged line 238
"DHX Scheduler (Project)"(Page 50621).LoadSchedulerSectionsPaginated line 20
"DHX Scheduler (Project)"(Page 50621)."DhxScheduler - ControlReady"(Trigger) line 32
```

This pinpointed the failure to the exact array-indexing statements above (relative line 43 of `AddAncestorsToTemp`) — no guesswork was needed to locate the defect once the stack trace was available, and no other array/list indexing elsewhere in the paginated builder needed to be suspected once this was confirmed.

8.3 Fix

`LastAncestorAtLevel` was changed from a fixed `array[60] of Code[20]` to a `Dictionary of [Integer, Code[20]]` — a data structure with no fixed ceiling by construction, eliminating this entire class of bug rather than guessing at a "safe enough" replacement bound:

AFTER THE FIX — `codeunit_50604_DHXDataHandler.al`, `AddAncestorsToTemp`

```
LastAncestorAtLevel: Dictionary of [Integer, Code[20]];
LastAncestorCode: Code[20];
...
if (JobTaskReal.Indentation > 0)
    and LastAncestorAtLevel.Get(JobTaskReal.Indentation, LastAncestorCode) then
    ParentOfMap.Add(JobTaskReal."Job Task No.", LastAncestorCode);
...
LastAncestorAtLevel.Set(JobTaskReal.Indentation + 1, JobTaskReal."Job Task No.");
```

`Dictionary.Get(key, var value)` returns a boolean and the looked-up value together, replacing the old array read plus blank-check in one call; `Dictionary.Set(key, value)` upserts (adds or overwrites) regardless of whether the key has been seen before, replacing the old unconditional indexer assignment. `Clear()` on a Dictionary variable — already used once per Job iteration to reset state between Jobs — works identically to `Clear()` on the old array, so that call site needed no change.

WHY THIS WAS CAUGHT BEFORE SIGN-OFF, NOT AFTER

This defect was found because the enhancement was re-tested live against a real, populous demo company (CRONUS NL) via Playwright immediately after implementation, rather than relying solely on `al_compile` / `al_build` succeeding and the new unit tests (which use a small synthetic fixture, and therefore never exercised a WBS deep enough to trip the bound) passing. Section 9 covers the full verification pass, including the confirmed-working re-test after this fix.

9. Verification

9.1 Static verification

Check	Result
<code>al_compile</code>	Succeeded, 0 errors
<code>al_build</code>	Succeeded, package generated
Codeunit 60029 "Task Scheduler Sections Perf" (3 tests, correctness-by-equivalence over a 2-job/3-level-WBS/one-zero-Day-Planning-job fixture)	3 passed, 0 failed, 0 skipped
<code>git diff --stat</code> on <code>src/dhx/ganttdemo2/page_50620_GanttDemo.al</code> and <code>codeunit_50613_GanttChartDataHandler.al</code>	Empty — confirmed untouched by this work
<code>GetWeekPeriodDates</code> signature (shared with Gantt and 3 other scheduler pages)	Unchanged — confirmed by diff inspection, not modified

9.2 Live browser verification (Playwright, Chromium, signed-in web client)

Scenario	Result
First live re-test, immediately after the Part A/B implementation (before Section 8's fix)	Crashed with the "Index out of bounds" error dialog documented in Section 8 — caught, not shipped
Initial open, after the Section 8 fix + republish	Renders correctly — full Job/Job Task section tree and Day Planning events for the current week, matching the page's known-good content (Work Order Demo, DJB0001 Riverside, DJB0002 Meadowbrook, etc.). No error dialog. No new browser console errors (only pre-existing, unrelated platform noise — an avatar-photo 404 and other-company OData 401s from the BC shell, present before this change too).
Click "Next"	Advances one week, section tree and events render correctly, no crash — confirms the shared <code>RefreshSchedule / LoadSchedulerSectionsPaginated</code> path (Section 4.2) works, not just the initial-open path.

NO ISOLATED BEFORE/AFTER TIMING BENCHMARK

As with the companion Gantt document, no controlled before/after latency measurement was captured for this change — obtaining one safely would require an isolated git worktree outside this OneDrive-synced project folder, which was judged unnecessary given the qualitative evidence: the two rewritten hot spots (Section 2) are structurally $O(\text{passes} \times \text{tasks} \times \text{query})$ and $O(\text{rows} \times 4 \text{ Gets})$ respectively, both collapsed to $O(\text{distinct Jobs})$ and $O(\text{rows})$ with prefetched $O(1)$ lookups — a categorical algorithmic improvement independent of any specific timing number, and orthogonal to the perceived-responsiveness benefit of Part B's pagination.

10. Object Inventory

Object	Type	Status	Role
Codeunit 50604 "DHX Data Handler"	Codeunit	MODIFIED	<code>AddAncestorsToTemp</code> rewritten (Dictionary-based single pass); N+1 Resource/Vendor/JobTask gets replaced with bulk prefetch; new <code>GetYUnitElementsJSON_Project_Paged</code> ; new shared filter helper; dead unused vars removed from both <code>..._Project_StartEnd</code> overloads. <code>GetWeekPeriodDates</code> untouched.
Page 50621 "DHX Scheduler (Project)"	Page	MODIFIED	<code>ControlReady</code> / <code>RefreshSchedule</code> / <code>OnTimelineNavigate</code> funnel through new shared <code>LoadSchedulerSectionsPaginated</code> ; new <code>OnPageBackgroundTaskCompleted</code> / <code>OnPageBackgroundTaskError</code> / <code>OnPollSectionsResult</code> triggers; staleness guard on scope match.
"DHXProjectScheduleAddin"	Control Add-in	MODIFIED	+1 event (<code>OnPollSectionsResult</code>), +3 procedures (<code>AppendSections</code> , <code>NotifySectionsTaskPending</code> , <code>StopSectionsPolling</code>)
wrapper.js (projectschedule)	Add-in script	MODIFIED	New <code>AppendSections</code> (in-place <code>y_unit</code> refresh, modeled on <code>ToggleCollapseExpandAllSections</code>); bounded poll timer matching the Gantt resource panel's 500 ms/60-attempt shape
Codeunit 50720 "Task Scheduler BG Sections"	Codeunit	NEW	Page Background Task target — builds the remaining-Jobs section/event JSON off the interactive request path
Test Codeunit 60029 "Task Scheduler Sections Perf"	Codeunit (Test)	NEW	3 correctness-by-equivalence tests over a multi-job, multi-depth-WBS, zero-Day-Planning-job fixture
Page 50620 "Gantt Demo DHX 2" / Codeunit 50613 "GanttChartDataHandler"	Page / Codeunit	unchanged	Explicitly out of scope — confirmed untouched (Section 9.1); Gantt shares only the unmodified, pure-date-math <code>GetWeekPeriodDates</code> procedure with this work

11. Known Limitations & Follow-ups

- **No isolated before/after benchmark** (Section 9.2) — same OneDrive-sync constraint documented in the companion Gantt document; not pursued for the same reason.
- **Pagination boundary is Job-granular, not row-granular.** A single Job with an unusually large number of section rows (well beyond the ~50-row target) will still render synchronously in full, since a Job is never split across the sync/background boundary (Section 4.1). This is a deliberate trade-off favoring a coherent first page over a strict row-count ceiling.
- **Poll ceiling is 30 s**, matching the Gantt resource panel's own limit and the background task's server-side timeout — if a background fetch of the remaining Jobs legitimately takes longer (very large remaining scope, server under load), the client stops polling before delivery; the user would need to trigger another reload (e.g. Refresh Data) to pick up the remainder.
- `GetYUnitElementsJSON_Project_Paged` **is self-contained** rather than delegating to the non-paged procedure (Section 5.1) — a deliberate choice to avoid destabilizing the already-verified non-paged path, at the cost of the two procedures' Part-A logic living in two places rather than one. Test codeunit 60029 exercises both to guard against them drifting apart.
- **Test fixture WBS depth is shallow** relative to the depth that actually triggered Section 8's bug — the fixture was not deepened to reproduce the exact crash after the fix, since the fix removes the entire class of bound-related failure rather than raising a ceiling; live verification against the real dataset that originally crashed is the evidence of record instead (Section 9.2).

Appendix A — Page Background Task API Reference

Identical platform mechanism documented in the companion Gantt document's Appendix A; reproduced here for a reader of this document alone.

Element	Signature / Note
<code>CurrPage.EnqueueBackgroundTask</code>	(var TaskId: Integer; CodeunitId: Integer; [Parameters: Dictionary of [Text, Text]], [Timeout: Integer], [ErrorLevel: PageBackgroundTaskErrorLevel])
<code>Page.GetBackgroundParameters()</code>	Called inside the background codeunit's <code>OnRun()</code> to read the input dictionary
<code>Page.SetBackgroundTaskResult(Result)</code>	Called inside <code>OnRun()</code> to return the output dictionary
<code>OnPageBackgroundTaskCompleted(TaskId; Results)</code>	Page trigger. Cannot call the control add-in directly (Section 7.1) — stash into plain AL vars only
<code>OnPageBackgroundTaskError(TaskId; ErrorCode; ErrorText; ErrorCallStack; var IsHandled)</code>	Page trigger, fired on failure/timeout of the background task
Session characteristics	Read-only (a write raises a permission error); own OpenCompany /transaction; canceled if the page closes; default 5 concurrent tasks per parent session; max 600,000 ms timeout

Appendix B — Glossary of Terms

Term	Meaning in this document
Section	One row of the DHTMLX Scheduler's timeline "y_unit" tree — a Job or a Job Task, nested to match the Job Task WBS hierarchy. Analogous to a Gantt task-tree row, but scoped to only the Job Tasks touched by a Day Planning in the visible week.
N+1 (query) problem	A pattern where fetching a list of N parent rows, then querying once per parent row for related data, produces N+1 (or more) total round trips instead of one bulk fetch. See Section 2's two hot spots.
Job boundary (pagination)	The rule that the first synchronous page always includes a Job's section rows in full or not at all — never mid-Job — so the background-loaded remainder never has to graft an orphaned sub-branch onto an already-rendered parent.
Stash-and-poll	The delivery pattern required because a Page Background Task's completion trigger cannot call the control add-in directly: the result is stashed into plain page variables, then delivered later from a normal, JS-pollled trigger. First established for the Gantt resource panel; reused unchanged here.
Staleness guard	The scope-match check in <code>OnPageBackgroundTaskCompleted</code> that discards a background result if the user has since navigated away from the period/filter it was computed for.
Dictionary of [Integer, Code[20]]	The unbounded replacement for the fixed <code>array[60] of Code[20]</code> that caused Section 8's production bug — an AL Dictionary keyed by an arbitrary integer level index has no ceiling to exceed.

Appendix C — How to Verify This Yourself

Every code excerpt in this document was copied from the working tree at the time of writing. Suggested reading order:

1. Open `src/codeunit/codeunit_50604_DHXDataHandler.al` and search for `LastAncestorAtLevel` — confirm it is declared as `Dictionary of [Integer, Code[20]]`, not an array (Section 8.3), and that every use is `.Get(...)` / `.Set(...)`, never `[...]` indexer syntax.
2. In the same file, search for `GetYUnitElementsJSON_Project_Paged` — confirm it accepts a `MaxRows: Integer` parameter and returns a `var RemainingJobFilter: Text` out-parameter (Section 4.1).
3. Open `src/dhx/projectschedule/projectschedule_50621_DHXScheduleBoard.al` and search for `LoadSchedulerSectionsPaginated` — confirm all three of `ControlReady`, `RefreshSchedule`, and `OnTimelineNavigate` call it (Section 6.1).
4. In the same file, search for `OnPageBackgroundTaskCompleted` — confirm it never references `CurrPage.DhxScheduler`, and that only `OnPollSectionsResult` does (Section 7.1).
5. Open `src/dhx/projectschedule/codeunit_50720_TaskSchedulerBGSections.al` in full — confirm it never calls anything on `CurrPage`, consistent with running in a separate, UI-less child session.
6. Open `src/dhx/projectschedule/wrapper.js` and search for `AppendSections` — confirm it mutates `y_unit / y_unit_original` and calls `onOptionsLoad`, and never calls `RecreateTimelineView`, `Init`, or `RefreshTimeline` (Section 7.2).
7. Run `test/TaskSchedulerSectionsPerf.Test.Codeunit.al` (codeunit 60029) via `al_run_tests` — confirm 3 passed, 0 failed.
8. Open Task Scheduler in a live Business Central web client against a company with a multi-level WBS and click Previous/Today/Next — confirm no "Index out of bounds" error and no console error beyond pre-existing platform noise.